



MapReduce Framework

University of Virginia

DS 7200: Distributed Computing

Last Updated: August 20, 2026

SOURCES

Hadoop: The Definitive Guide, Tom White

OBJECTIVES

- Introduction to the MapReduce framework

CONCEPTS

- MapReduce
 - Jobs
 - Tasks
 - Map operation
 - Reduce operation
 - Shuffle operation
-

Introducing MapReduce

MapReduce is a programming framework for processing large data sets with a parallel, distributed algorithm on a cluster.

First the data set is "mapped" into a collection of `<key, value>` pairs, and then "reduced" over all pairs with the same key.

We need to discuss what this means.

The operations are surprisingly broad, enabling them to handle a wide range of use cases.

The popular minimal example is to do a word count on some text.

Initially published in 2004 by employees at Google

A `MapReduce` job is a unit of work the client wants to be performed

A job consists of:

- Input data
- `MapReduce` program
- configs

The job will be divided into *tasks*

Two types of tasks: *map tasks* and *reduce tasks*

Aside on `Hadoop`

`MapReduce` forms the computation paradigm for Hadoop

`Hadoop` was created by Doug Cutting and Mike Cafarella. Version 1 was called Nutch.

Yahoo! was instrumental in providing a dedicated team and resources to turn Hadoop into a system that ran at web scale.

Hadoop is developed and maintained by the Apache Software Foundation.

Aside on Hadoop and Spark

Spark does not follow the `MapReduce` framework in the same way that `Hadoop` does, where `Mapper` and `Reducer` functions are detailed explicitly. This is actually a big advantage of Spark.

Spark and Hadoop can be better together. For example, some teams will use Hadoop data storage (HDFS) with Spark.

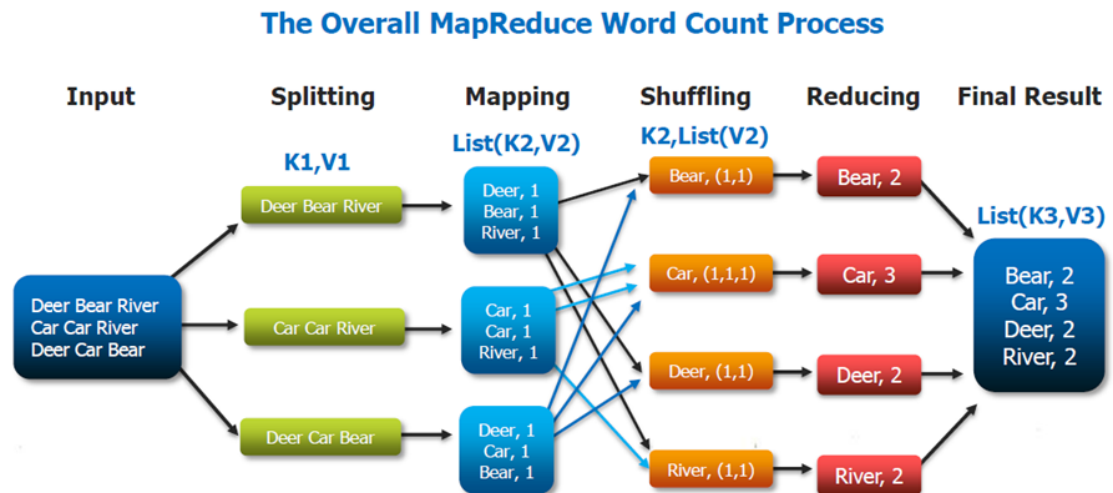
That is, a Spark job can read data from HDFS, and write results to HDFS.

Reported runtimes: Spark can be as much as 10 times faster than `MapReduce` for batch processing, and up to 100 times faster for in-memory analytics.

A large fraction of this course will focus on Spark for various tasks.

Next we illustrate the **MapReduce** framework with an example.

MapReduce Word Count Process Diagram



Some Things to Notice:

Splitting Step

A given number of records is assigned to each worker

NOTE:

More splits means less processing time per split

However, the overhead of managing splits and **map** task creation starts to dominate total job execution time

Mapping Step

Each token (word) is given a value of 1 representing the count.

The token, count forms a **<key, value> pair**.

If the same pair appears n times on a worker, there would be n entries of **<map, 1>**. In other words, no aggregation is done at this step (it happens at the **reduce** step).

Shuffling Step

The **<key, value>** pairs are moved across machines at this point, so that pairs with the same key are gathered on the same machine. This is a costly step in the process.

Reducing Step

A **reducer** applies a given operation over the values by key. For this word count example, the counts would be summed for each key and stored with the key.

Map and Reduce in General

Map

Higher-order function that applies a function to each element of a list (e.g., $x \Rightarrow x * 2$)

Reduce

Higher-order function that applies a combining operation (e.g., cumulative sum of values)

Combiner Functions

It can be expensive to shuffle data within the cluster, and the limiting factor is the bandwidth on the cluster.

Combiner Functions can be run on the map output, before the shuffle occurs. This acts locally on each node, and reduces some of the data transfer. Note this won't always work (e.g., can't be used for computing the average, but CAN be used for computing the maximum).

Word Count Example

Data source: Spark README file

```
In [1]: # import libraries
        from pyspark.sql import SparkSession
        import os

        # context manager: minimal settings
        spark = SparkSession.builder.getOrCreate()

        sc = spark.sparkContext
```

```

WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.p
roperties
26/09/04 13:08:58 WARN Utils: Your hostname, Sabines-MacBook-Pro.local,
resolves to a loopback address: 127.0.0.1; using 172.25.149.14 instead
(on interface en0)
26/09/04 13:08:58 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to
another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.p
roperties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use s
etLogLevel(newLevel).
/Users/sabinesegaloff/Code/UVA_Fall_26/distributed_computing/.venv/lib/
python3.13/site-packages/pyspark/testing/utils.py:127: FutureWarning: P
ySpark does not yet fully support pandas >= 3.0.0. Some features may no
t work correctly. It is recommended to use pandas < 3.0.0 for now.
    require_minimum_pandas_version()
26/09/04 13:08:59 WARN NativeCodeLoader: Unable to load native-hadoop l
ibrary for your platform... using builtin-java classes where applicable
26/09/04 13:09:00 WARN Utils: Service 'SparkUI' could not bind on port
4040. Attempting port 4041.
26/09/04 13:09:00 WARN Utils: Service 'SparkUI' could not bind on port
4041. Attempting port 4042.

```

In [2]: spark

Out[2]: **SparkSession - in-memory**

SparkContext

Spark UI

Version	v4.2.0
Master	local[*]
AppName	pyspark-shell

In [3]: *# list files in the directory*
os.listdir()

Out[3]: ['uva_seal.png',
'map_reduce_framework.ipynb',
'map_reduce_example2.png',
'README.txt',
'map_reduce_framework.html']

In [4]: os.getcwd()

Out[4]: '/Users/sabinesegaloff/Code/UVA_Fall_26/distributed_computing/01_big_
data_and_distributed_systems/content_mapreduce'

```
In [5]: # Read in data
lines = sc.textFile("README.txt")
```

```
In [6]: type(lines)
```

```
Out[6]: pyspark.core.rdd.RDD
```

```
In [7]: lines.count()
```

```
Out[7]: 41
```

```
In [8]: # show some records
lines.take(5)
```

```
Out[8]: ['Apache Spark',
        'Spark is a unified analytics engine for large-scale data processing. It provides high-level APIs in Scala, Java, Python, and R, and an optimized engine that supports general computation graphs for data analysis. It also supports a rich set of higher-level tools including Spark SQL for SQL and DataFrames, MLlib for machine learning, GraphX for graph processing, and Structured Streaming for stream processing.',
        '',
        'https://spark.apache.org/']
```

Show top 10 most frequent words (we will examine in detail later)

```
In [9]: words = lines.flatMap(lambda x: x.split())
wordcounts = words.map(lambda x: (x, 1)) \
                  .reduceByKey(lambda x,y:x+y) \
                  .map(lambda x:(x[1],x[0])) \
                  .sortByKey(False)
wordcounts.take(10)
```

```
Out[9]: [(13, 'the'),
        (11, 'Spark'),
        (7, 'to'),
        (6, 'for'),
        (6, 'and'),
        (6, 'a'),
        (6, 'can'),
        (6, 'run'),
        (5, 'using'),
        (5, 'is')]
```

Question: How are `map()` and `flatMap()` different?

```
In [10]: # Distribute data to the workers
data = sc.parallelize([3,4,5])
```

```
type(data)
```

```
Out[10]: pyspark.core.rdd.RDD
```

```
In [11]: sc.parallelize([3,4,5]).map(lambda x: [x, x*x]).collect()
```

```
Out[11]: [[3, 9], [4, 16], [5, 25]]
```

```
In [12]: sc.parallelize([3,4,5]).flatMap(lambda x: [x, x*x]).collect()
```

```
Out[12]: [3, 9, 4, 16, 5, 25]
```

map: a list of lists

flatMap: a flattened list

TRY FOR YOURSELF (UNGRADED EXERCISES)

1. Copy the word count code into the cell below. Modify it to print the top 20 wordcount pairs.

```
In [15]: words = lines.flatMap(lambda x: x.split())
wordcounts = words.map(lambda x: (x, 1)) \
                  .reduceByKey(lambda x,y:x+y) \
                  .map(lambda x:(x[1],x[0])) \
                  .sortByKey(False)
wordcounts.take(20)
```

```
Out[15]: [(13, 'the'),
          (11, 'Spark'),
          (7, 'to'),
          (6, 'for'),
          (6, 'and'),
          (6, 'a'),
          (6, 'can'),
          (6, 'run'),
          (5, 'using'),
          (5, 'is'),
          (4, 'with'),
          (4, '*'),
          (4, 'examples'),
          (4, 'in'),
          (4, 'also'),
          (4, 'You'),
          (3, 'an'),
          (3, 'you'),
          (3, 'use'),
          (3, 'including')]
```

2. Copy the word count code into the cell below. Modify it to filter out rows containing the word *Spark* and then execute the word count, returning the top 10 wordcount pairs.

```
In [ ]: filtered_lines = lines.filter(lambda line: "Spark" not in line)
words = filtered_lines.flatMap(lambda x: x.split())
wordcounts = words.map(lambda x: (x, 1)) \
                  .reduceByKey(lambda x, y: x+y) \
                  .map(lambda x: (x[1], x[0])) \
                  .sortByKey(False)
wordcounts.take(10)
```

```
Out[ ]: [(7, 'the'),
(6, 'to'),
(5, 'run'),
(4, '*'),
(4, 'can'),
(3, 'you'),
(3, 'examples'),
(3, 'if'),
(3, 'a'),
(2, 'command, ')]
```

SOLUTIONS

```
In [13]: # 1)

wordcounts = lines.flatMap(lambda x: x.split()) \
                  .map(lambda x: (x, 1)) \
                  .reduceByKey(lambda x, y: x+y) \
                  .map(lambda x: (x[1], x[0])) \
                  .sortByKey(False)

wordcounts.take(20)
```

```
Out[13]: [(13, 'the'),
          (11, 'Spark'),
          (7, 'to'),
          (6, 'for'),
          (6, 'and'),
          (6, 'a'),
          (6, 'can'),
          (6, 'run'),
          (5, 'using'),
          (5, 'is'),
          (4, 'with'),
          (4, '*'),
          (4, 'examples'),
          (4, 'in'),
          (4, 'also'),
          (4, 'You'),
          (3, 'an'),
          (3, 'you'),
          (3, 'use'),
          (3, 'including')]
```

```
In [14]: # 2)
```

```
wordcounts = lines.filter(lambda x: "Spark" not in x) \
    .flatMap(lambda x: x.split()) \
    .map(lambda x: (x, 1)) \
    .reduceByKey(lambda x,y:x+y) \
    .map(lambda x:(x[1],x[0])) \
    .sortByKey(False)

wordcounts.take(10)
```

```
Out[14]: [(7, 'the'),
          (6, 'to'),
          (5, 'run'),
          (4, '*'),
          (4, 'can'),
          (3, 'you'),
          (3, 'examples'),
          (3, 'if'),
          (3, 'a'),
          (2, 'command,')]
```